

EECS 348 Assignment 1: GenAI Code Comparison Analysis

Josselyn T. Bui

3144923

- a. GenAI Programs Used: ChatGPT (OpenAI) and Gemini (Google)
- b. How I accessed them: via web version, no APIs/IDE plugins were used.
- c. Prompt used: Write a C program that picks a secret number between 1 and 10 (use a fixed number like 7). The user gets up to 3 tries to guess the number. Use printf to prompt the user and scanf to read their guess. Use a while or for loop to let the user guess up to 3 times. Use if-else statements to check if the guess is too high, too low, or correct. Stop the loop immediately if the user guesses the correct number. Print a final win or lose message.
- d. Generated Code:

ChatGPT:

```
#include <stdio.h>
```

```
int main()
{
    int secret = 7;
    int guess;
    int tries = 0;

    while (tries < 3)
    {
        printf("Guess the secret number (1-10): ");
        scanf("%d", &guess);

        tries++;

        if (guess == secret)
        {
            printf("Correct! You win!\n");
            break;
        }
        else if (guess < secret)
        {
            printf("Too low!\n");
        }
        else
```

```

        {
            printf("Too high!\n");
        }
    }

    if (guess != secret)
    {
        printf("Sorry, you lose! The secret number was %d.\n", secret);
    }

    return 0;
}

```

Gemini output:

```
#include <stdio.h>
```

```

int main() {
    const int secret_number = 7;
    int guess;
    int guessed_correctly = 0;

    printf("Welcome to the Number Guessing Game!\n");
    printf("I'm thinking of a number between 1 and 10.\n");
    printf("You have up to 3 tries to guess it.\n\n");

    for (int attempt = 1; attempt <= 3; attempt++) {
        printf("Attempt %d: Enter your guess: ", attempt);
        scanf("%d", &guess);

        if (guess == secret_number) {
            printf("Congratulations! You guessed the correct number!\n");
            guessed_correctly = 1;
            break; // Stop the loop immediately
        } else if (guess < secret_number) {
            printf("Too low!\n\n");
        } else {
            printf("Too high!\n\n");
        }
    }
}

```

```

// Final outcome check
if (!guessed_correctly) {
    printf("Sorry, you've run out of tries! The secret number was %d.\n", secret_number);
}

return 0;
}

```

Analysis:

Correctness: Both programs actually do satisfy what the assignment asks for, so in terms of raw functionality they're both correct. Where they differ is in how each one figures out whether the user actually won. ChatGPT's version relies on the fact that after the while loop ends, the guess variable still holds whatever the user last typed in, so it just checks `guess != secret` one more time outside the loop to decide the outcome. This works fine as long as `scanf` always successfully reads a number, but it's a little fragile, because if `scanf` ever failed to parse the input, `guess` wouldn't get properly updated and would just be left holding whatever value it had before, which could then throw off that final check. Gemini's version avoids this problem entirely by introducing a separate variable called `guessed_correctly`, which starts at 0 and only gets set to 1 the moment the user's guess matches the secret number inside the loop. So instead of trying to reconstruct what happened after the fact, it just tracks the outcome directly as it happens, which is a safer way to do it. That said, neither program actually checks whether `scanf` succeeded in the first place, so if a user types a letter instead of a number, both programs have the same underlying gap in correctness.

Execution Time: In terms of execution time, there's really no meaningful difference between the two. Both programs are bounded to a maximum of 3 loop iterations, and each iteration is just doing a `printf`, a `scanf`, and a couple of integer comparisons, so the number of operations stays constant no matter what. Since the loop can only ever run 3 times at most, both programs are $O(1)$ in terms of time complexity, and any actual difference in how long the program takes to finish is going to come down to how long the user takes to type their guess, not anything happening in the code itself.

Space Complexity: Space complexity is also $O(1)$ for both, since neither program uses any data structures that grow with input size, just a handful of individual variables. ChatGPT's version declares three integers total: `secret`, `guess`, and `tries`. Gemini's version declares four: `secret_number`, `guess`, `guessed_correctly`, and the `attempt` variable that's local to the for loop. So Gemini's version does use one more integer on the stack, but since we're only talking about a single extra 4 byte variable, this difference is negligible and doesn't have any real practical impact on the program's memory usage.

Maintainability: This is where the two programs mainly are different. Gemini's version is more maintainable for a few reasons. First, the variable naming is more descriptive, using `secret_number` instead of just `secret`, which makes it clearer what the variable represents at a glance. Second, using the `guessed_correctly` flag makes the win condition explicit and easy to follow, since you can just look at where the flag gets set to know exactly when and why the user won, instead of having to reason backwards through the loop's exit state. Third, Gemini's output already includes the attempt number in its prompt, which lines up more closely with the format shown in the assignment's example output, so less restructuring is needed to match it. ChatGPT's version is slightly shorter, but that comes at the cost of readability, because to actually understand why `guess != secret` correctly reflects the final outcome after the loop has already ended, you have to mentally trace through every iteration of the loop and think about what state `guess` would be left in, which makes the logic less obvious to maintain or extend later on.

Selection and Justification: I selected Gemini's version as the basis for this assignment. Its use of an explicit `guessed_correctly` flag is a safer and clearer correctness pattern than relying on the final state of a loop variable, and its overall structure and output format were already closer to the assignment's example output. These qualities made it a stronger and more maintainable starting point than ChatGPT's version.

Improvements made:

Correctness: Added validation of `scanf`'s return value so non-numeric input (like a stray letter) gets caught and reported instead of silently leaving `guess` with an undefined or leftover value. The bad input also gets cleared from the buffer afterward so it doesn't mess up the next read. This fixes the gap that was present in both original programs.

Execution time: No change needed. The program is already $O(1)$ since the loop is capped at 3 iterations, and the runtime is dominated by how long the user takes to type their guess, not by the code itself.

Space complexity: No change needed. The program already uses a small, fixed number of variables, so it's $O(1)$ and appropriate for the scope of this assignment.

Maintainability: Rewrote all the prompts and messages to match the assignment's exact example output format, like using "Attempt 1/3. Enter your guess: " instead of Gemini's more generic wording. Also added a full prologue comment block plus block- and line-level comments throughout, each labeled to show whether that part was written by me or adapted from Gemini's output, per the assignment's requirements.